📦 Dane w skali — formaty i przechowywanie danych

Zanim odpalisz klaster: dlaczego CSV to cichy wróg Big Data

Python w Zastosowaniach Big Data | Wykład 2

Prowadzący: Pavlo Melnyk

📋 Plan wykładu

1. Kiedy dane przestają być "małe"? Granice jednej maszyny
2. Formaty wierszowe vs kolumnowe — CSV, JSON, Avro, **Parquet**, **Arrow**
3. Kompresja i typy danych — jak zmieścić 10× więcej w tym samym RAM-ie
4. **Partycjonowanie** — jak nie czytać danych, których nie potrzebujesz
5. Przetwarzanie równoległe w Pythonie — *divide & conquer*
6. Gdzie trzymać dane: **Data Lake**, **Data Warehouse**, **Lakehouse**

> 🎯 **Po tym wykładzie:** Będziesz wiedzieć, jak zapisać dane tak, żeby Spark, BigQuery i Twój laptop Ci za to podziękowały.

🔄 Szybkie przypomnienie

Na Wykładzie 1 robiliśmy **EDA** na 1 200 maratończykach:

```
df = pd.read_csv("maratony.csv")   # 👈 ~100 KB, wczytuje się w mgnieniu oka
df.describe()
```

A co, jeśli zamiast 1 200 wierszy masz **1,2 miliarda** zdarzeń z zegarków sportowych? ⌚

> 🤔 Ten sam kod. Ten sam laptop. Ile to potrwa? **Czy w ogóle się uda?**

💡 Dlaczego to ważne?

W typowym pipeline Big Data **najdroższe nie jest liczenie — tylko czytanie danych** (I/O).

Firma	Co zrobiła	Efekt
Uber	Przejście na Parquet + partycjonowanie	Zapytania szybsze o rzędy wielkości
Netflix	Apache Iceberg na Parquet (Data Lakehouse)	Petabajty danych z transakcjami ACID
Twitter/X	Format kolumnowy dla logów	Wielokrotnie mniej danych na dysku

> 💸 W chmurze (BigQuery, Athena) płacisz **za przeskanowane bajty**. Zły format = wyższy rachunek!

📏 Kiedy dane przestają być "małe"?

Rozmiar danych	Co zwykle wystarczy	Uwagi
< 1 GB	pandas	Komfortowo w RAM
1–50 GB	pandas + Parquet, chunki, DuckDB, Polars	Jedna maszyna, sprytne I/O
50 GB – kilka TB	Spark na klastrze / BigQuery	Skalowanie wszerz
> kilka TB	Spark, BigQuery, Data Lakehouse	Bez rozproszenia ani rusz

> ⚠️ **Reguła kciuka:** pandas potrzebuje często **5–10× więcej RAM** niż rozmiar pliku CSV na dysku!

🤔 Pomyśl przez chwilę

Masz **plik CSV 40 GB** z logami sklepu internetowego i laptop z **16 GB RAM**.

Szef pyta: "*Ile zarobiliśmy w październiku w Krakowie?*"

- Czy `pd.read_csv("logi.csv")` zadziała? 💥
- Czy musisz wczytać **wszystkie** kolumny?
- Czy musisz wczytać **wszystkie** miesiące i miasta?

> 💬 Zapamiętaj odpowiedzi — dziś poznamy 3 techniki, które rozwiązują ten problem.

🧱 Anatomia pliku: wiersze vs kolumny

Ta sama tabela, dwa sposoby zapisu na dysku:

```
Tabela:  id | miasto  | kwota
          1 | Kraków  | 12.5
          2 | Gdańsk  |  8.0
          3 | Kraków  | 30.0

Wierszowo (CSV, JSON, Avro):   1,Kraków,12.5 | 2,Gdańsk,8.0 | 3,Kraków,30.0
Kolumnowo (Parquet, ORC):      1,2,3 | Kraków,Gdańsk,Kraków | 12.5,8.0,30.0
```

- **Wierszowo** → szybki zapis pojedynczego rekordu (OLTP, aplikacje)
- **Kolumnowo** → szybka analiza wybranych kolumn (OLAP, analityka)

> 💡 `SUM(kwota)` w formacie kolumnowym czyta **tylko jedną kolumnę**!

📄 CSV — stary znajomy

Zalety ✅

- Czytelny dla człowieka, otworzysz go w Excelu
- Uniwersalny — każde narzędzie go obsługuje

Wady ❌

- **Brak schematu** — typy trzeba zgadywać (`inferSchema` = czytanie danych 2×)
- Liczby zapisane jako tekst → **duże pliki**
- Trzeba czytać **całe wiersze**, nawet gdy potrzebujesz jednej kolumny
- Problemy z przecinkami, cudzysłowami, kodowaniem znaków 😵

> 🎯 CSV jest świetny do **wymiany** danych, fatalny do **analizy** w skali.

📄 JSON i JSON Lines

```python
import pandas as pd

# JSON Lines – jeden obiekt JSON na linię (popularne w logach i API)
df = pd.read_json("zdarzenia.jsonl", lines=True)
print(df.head())
```

```
{"user": 17, "event": "click", "ts": "2026-10-13T10:00:01"}
{"user": 42, "event": "buy", "ts": "2026-10-13T10:00:03", "kwota": 99.9}
```

- ✅ Elastyczny — obsługuje zagnieżdżenia i brakujące pola
- ❌ Klucze powtarzane w **każdym** rekordzie → jeszcze większe pliki niż CSV

🏆 Apache Parquet — król analityki

Parquet = otwarty, **kolumnowy** format plików stworzony dla Big Data (Twitter + Cloudera, 2013).

- 📐 **Schemat zapisany w pliku** — typy kolumn są znane od razu
- 🗜️ **Kompresja per kolumna** — podobne wartości kompresują się świetnie
- 📊 **Statystyki min/max** dla bloków danych (*row groups*)
- 🔍 **Column pruning** — czytasz tylko potrzebne kolumny
- 🚫 **Predicate pushdown** — pomijasz bloki, które na pewno nie pasują do filtra

> 🌍 Domyślny format dla Spark, BigQuery (import), Delta Lake, Iceberg, DuckDB, Polars.

🔍 Parquet w akcji — column pruning

```
import pandas as pd

df.to_parquet("przejazdy.parquet")  # 👈 wymaga pyarrow (jest w Colab)

# Wczytaj TYLKO dwie kolumny — reszta nawet nie zostanie odczytana z dysku
kwoty = pd.read_parquet(
    "przejazdy.parquet",
    columns=["miasto", "kwota_pln"],
)
print(kwoty.groupby("miasto")["kwota_pln"].sum())
```

> 💡 Przy 50 kolumnach i potrzebie 2 → czytasz ~**4%** danych. To jest ta magia!

🚫 Predicate pushdown — pomijanie bloków

Parquet dzieli dane na **row groups** i zapisuje dla nich statystyki:

```
Row group 1: kwota min=0.5   max=48.0
Row group 2: kwota min=12.0  max=95.0
Row group 3: kwota min=101.0 max=480.0    👈 tylko ten blok ma kwoty > 100
```

```
duze = pd.read_parquet(
    "przejazdy.parquet",
    filters=[("kwota_pln", ">", 100)],  # 👈 filtr "wypchnięty" do czytnika
)
```

> 🧠 Czytnik patrzy na min/max i **w ogóle nie dotyka** bloków 1 i 2.

⚡ Apache Arrow — kolumny w pamięci RAM

Parquet = format kolumnowy **na dysku**. **Arrow** = format kolumnowy **w pamięci**.

```
import pyarrow.parquet as pq

tabela = pq.read_table("przejazdy.parquet")   # 👈 pyarrow.Table (Arrow)
print(tabela.schema)
df = tabela.to_pandas()                        # konwersja do pandas
```

- 🔁 **Zero-copy** — Spark ↔ pandas ↔ DuckDB ↔ Polars bez kopiowania danych
- 🚀 To dzięki Arrow działa szybkie `toPandas()` i pandas UDF w PySpark
- 🐼 pandas 2.x może używać Arrow jako backendu: `dtype_backend="pyarrow"`

🦅 Avro — wierszowy, ale z głową

Apache Avro = binarny format **wierszowy** ze schematem (JSON).

- ✅ Szybki zapis rekord po rekordzie → idealny dla **strumieni** (Kafka!)
- ✅ **Schema evolution** — dodasz nową kolumnę, stare dane dalej się czytają
- ❌ Do analityki kolumnowej słabszy niż Parquet

> 🔮 Spotkamy go ponownie na Wykładzie 12 przy **Kafce i streamingu**.

Porównanie formatów

Cecha	CSV	JSON	Avro	Parquet
Układ	wierszowy	wierszowy	wierszowy	**kolumnowy**
Schemat	❌	❌	✅	✅
Kompresja	słaba	słaba	dobra	**świetna**
Czytelny dla człowieka	✅	✅	❌	❌
Szybki zapis rekordów	✅	✅	✅	❌
Szybka analityka	❌	❌	–	✅
Typowe użycie	wymiana	API, logi	Kafka	**Data Lake**

Kompresja — nie wszystkie są równe

Kodek	Stopień kompresji	Szybkość	Uwagi
snappy	średni	⚡⚡⚡	Domyślny w Parquet i Spark
zstd	wysoki	⚡⚡	Świetny kompromis, coraz popularniejszy
gzip	wysoki	⚡	Wolny; `.csv.gz` **nie jest dzielony** (splittable)
brak	—	⚡⚡⚡	Duże pliki, dużo I/O

> ⚠️ Plik `logi.csv.gz` 40 GB czyta **jeden** worker Sparka — bo gzipa nie da się podzielić na kawałki. Parquet + snappy/zstd kompresuje **wewnątrz** bloków, więc nadal da się go dzielić!

Demo: CSV vs Parquet

```
import time
from pathlib import Path

df.to_csv("dane.csv", index=False)
df.to_parquet("dane.parquet", compression="snappy")

for plik in ["dane.csv", "dane.parquet"]:
    mb = Path(plik).stat().st_size / 1e6
    start = time.perf_counter()
    _ = pd.read_csv(plik) if plik.endswith("csv") else pd.read_parquet(plik)
    print(f"{plik:14} {mb:7.1f} MB  {time.perf_counter() - start:.2f} s")
# dane.csv          38.2 MB  1.41 s
# dane.parquet       9.7 MB  0.12 s   👈 ~4× mniej miejsca, ~10× szybciej
```

🧠 Typy danych = pamięć

```
print(df.memory_usage(deep=True).sum() / 1e6)  # 👈 np. 112.4 MB

df["miasto"] = df["miasto"].astype("category")       # kilka unikalnych wartości
df["liczba_osob"] = pd.to_numeric(df["liczba_osob"], downcast="integer")

print(df.memory_usage(deep=True).sum() / 1e6)  # 👈 np. 41.0 MB
```

- `object` (napisy) → `category`, gdy wartości się powtarzają
- `int64` → `int8` / `int16`, gdy zakres jest mały
- `float64` → `float32`, gdy nie potrzebujesz 15 cyfr precyzji

> 💡 Parquet **zapamiętuje** typy (także `category`) — CSV zapomina je przy każdym zapisie.

🍰 Czytanie w kawałkach (chunks)

Plik większy niż RAM? Przetwarzaj go **kawałek po kawałku**:

```
suma_per_miasto: dict[str, float] = {}

for chunk in pd.read_csv("logi.csv", chunksize=1_000_000):
    czesciowe = chunk.groupby("miasto")["kwota_pln"].sum()     # 👈 "map"
    for miasto, kwota in czesciowe.items():
        suma_per_miasto[miasto] = suma_per_miasto.get(miasto, 0) + kwota  # 👈 "reduce"

print(suma_per_miasto)
```

> 🔮 Zauważ wzorzec: **policz częściowo** → **połącz wyniki**. To dokładnie idea **MapReduce** z Wykładu 3!

Partycjonowanie — katalogi zamiast jednego pliku

```
przejazdy/
├── miasto=Gdańsk/
│   └── part-0.parquet
├── miasto=Kraków/
│   └── part-0.parquet
└── miasto=Warszawa/
    └── part-0.parquet
```

- Układ **Hive-style**: `kolumna=wartość/` — rozumieją go Spark, BigQuery, DuckDB, pandas
- Zapytanie o Kraków czyta **tylko** katalog `miasto=Kraków/` → **partition pruning**
- Typowe klucze partycji: **data** (`rok=/miesiac=/dzien=`), region, kraj

📂 Partycjonowanie w pandas

```
df.to_parquet(
    "przejazdy/",
    partition_cols=["miasto"],   # 👈 jeden podkatalog na miasto
)

krakow = pd.read_parquet(
    "przejazdy/",
    filters=[("miasto", "==", "Kraków")],  # 👈 czyta tylko jedną partycję
)
print(len(krakow))
```

To samo w PySpark (poznasz na Wykładzie 5):

```
df.write.partitionBy("miasto").parquet("przejazdy/")
```

⚠️ Pułapka: problem małych plików

Small files problem — tysiące malutkich plików zamiast kilku dużych.

- Każdy plik = osobne otwarcie, metadane, zapytanie do storage'u 🐌
- HDFS NameNode trzyma metadane każdego pliku **w RAM** (Wykład 3!)
- Spark tworzy co najmniej jedno zadanie (task) na plik

> 🎯 **Złota zasada:** celuj w pliki rzędu **128 MB – 1 GB**. Nie partycjonuj po kolumnie z milionem unikalnych wartości (np. `user_id`)!

Przetwarzanie równoległe w Pythonie

Jeden rdzeń CPU = jeden kawałek naraz. Masz ich 8? Użyj wszystkich!

```
from concurrent.futures import ProcessPoolExecutor
from pathlib import Path

def suma_partycji(sciezka: Path) -> float:
    return pd.read_parquet(sciezka)["kwota_pln"].sum()

pliki = sorted(Path("przejazdy").rglob("*.parquet"))
with ProcessPoolExecutor() as pool:
    wyniki = list(pool.map(suma_partycji, pliki))   # 👈 równolegle!

print(f"Suma: {sum(wyniki):,.2f} PLN")
```

> Procesy zamiast wątków — omijamy **GIL** (Global Interpreter Lock).

📈 Scale up vs scale out

	Scale up (w górę)	Scale out (wszerz)
Pomysł	Mocniejsza maszyna	Więcej zwykłych maszyn
Przykład	Laptop → serwer 1 TB RAM	100 węzłów klastra
Limit	Fizyka i budżet 💸	Praktycznie brak
Awaria	Jedna maszyna = wszystko leży	Inne węzły przejmują pracę
Narzędzia	pandas, DuckDB, Polars	Hadoop, Spark, BigQuery

> 🍕 **Analogia:** Jeden kucharz z coraz większym piecem vs. sieć pizzerii. Przy 10 000 pizz dziennie wygrywa sieć!

🏛️ Gdzie trzymać dane?

Rozwiązanie	Dane	Schemat	Przykłady
Baza OLTP	bieżące transakcje	sztywny	PostgreSQL, MySQL
Data Warehouse	oczyszczone, do raportów	*schema-on-write*	BigQuery, Snowflake
Data Lake	wszystko, surowe pliki	*schema-on-read*	GCS, S3, HDFS
Lakehouse	pliki + transakcje ACID	oba	Delta Lake, Iceberg

> 💡 **Schema-on-write** = sprawdzasz schemat przy zapisie. **Schema-on-read** = interpretujesz dane dopiero przy odczycie.

🌊 Data Lake — jezioro danych

- Tani **object storage** (Google Cloud Storage, Amazon S3) — płacisz grosze za GB
- Wrzucasz **wszystko**: CSV, JSON, Parquet, zdjęcia, logi
- Przetwarzasz Sparkiem, BigQuery, Dataproc...

```
# pandas czyta bezpośrednio z GCS (wymaga pakietu gcsfs)
df = pd.read_parquet("gs://moj-bucket/przejazdy/miasto=Kraków/")
```

> 🐊 **Uwaga na "data swamp"** — jezioro bez katalogu, opisu i porządku zamienia się w bagno, w którym nikt nic nie znajdzie.

🏠 Data Lakehouse — najlepsze z dwóch światów

Lakehouse = pliki Parquet w Data Lake + **warstwa transakcyjna** (log zmian):

- ✅ Transakcje **ACID** — brak "pół-zapisanych" danych
- ✅ **UPDATE / DELETE / MERGE** na plikach (np. RODO: usuń dane klienta!)
- ✅ **Time travel** — zapytanie o dane "z wczoraj"
- ✅ Ewolucja schematu

```
# Delta Lake w PySpark – wygląda prawie jak zwykły Parquet
df.write.format("delta").mode("overwrite").save("gs://bucket/przejazdy_delta")
```

🥉🥈🥇 Architektura medalionowa



- **Bronze** — kopia 1:1 ze źródeł, nic nie poprawiamy
- **Silver** — typy, braki, duplikaty, łączenie tabel
- **Gold** — tabele "pod pytanie biznesowe"

> 🎯 W projekcie zaliczeniowym też możesz myśleć w ten sposób: surowe → czyste → wnioski!

🦆 Teaser: SQL bezpośrednio na plikach

```
import duckdb

wynik = duckdb.sql("""
    SELECT miasto, ROUND(SUM(kwota_pln), 2) AS przychod
    FROM 'przejazdy/**/*.parquet'
    GROUP BY miasto
    ORDER BY przychod DESC
""").df()
print(wynik)
```

> 💡 Bez bazy danych, bez serwera — SQL wprost na Parquecie. Ta sama idea stoi za **Spark SQL** (Wykład 5) i **BigQuery** (Wykład 11).

⚠️ Najczęstsze błędy

1. ❌ Trzymanie danych analitycznych w **CSV** "bo tak było zawsze"
2. ❌ `SELECT *` / wczytywanie wszystkich kolumn, gdy potrzebujesz dwóch
3. ❌ Partycjonowanie po kolumnie o **wysokiej kardynalności** → miliony małych plików
4. ❌ Jeden ogromny `.csv.gz` — nie da się go czytać równolegle
5. ❌ Typy `object` i `float64` "na wszelki wypadek" → 5× więcej RAM

✅ Dobre praktyki

1. ✅ **Parquet + snappy/zstd** jako domyślny format do analityki
2. ✅ Partycjonuj po kolumnach, po których **filtrujesz** (najczęściej data)
3. ✅ Pliki **128 MB – 1 GB** — ani za małe, ani za duże
4. ✅ Wczytuj tylko potrzebne kolumny i wiersze (`columns=` , `filters=`)
5. ✅ Dbaj o typy: `category` , `int32` , `float32`
6. ✅ Dokumentuj Data Lake — katalog, schematy, właściciel danych

📝 Podsumowanie — kluczowe wnioski

1. ✅ **I/O to wąskie gardło** — format danych decyduje o szybkości i kosztach
2. ✅ **Kolumnowy Parquet** > CSV do analityki: schemat, kompresja, column pruning, predicate pushdown
3. ✅ **Arrow** = kolumny w RAM, wspólny język Spark/pandas/DuckDB
4. ✅ **Partycjonowanie** pozwala nie czytać danych, których nie potrzebujesz
5. ✅ **Divide & conquer** (chunki, procesy) to zalążek MapReduce i Sparka
6. ✅ **Data Lake → Lakehouse** — nowoczesny sposób przechowywania Big Data

🔮 Co dalej?

Następny wykład (Wykład 3, 20.10.2026): Big Data i ekosystem **Hadoop** — co zrobić, gdy dane nie mieszczą się już na **jednej** maszynie? HDFS, MapReduce, YARN 🐘

Ćwiczenie:

- Laboratorium 2 — CSV vs Parquet, partycjonowanie i równoległe przetwarzanie danych z wypożyczalni hulajnóg w Google Colab 🛴

Do przeczytania:

- Apache Parquet — dokumentacja
- pandas — Scaling to large datasets

❓ Pytania?

Dziękuję za uwagę! 🎉

Pytanie na rozgrzewkę przed ćwiczeniami:

> Masz tabelę z 200 kolumnami i 5 lat danych. Analityk zawsze pyta o **3 kolumny** z **ostatniego miesiąca**.
> Jak zapiszesz te dane, żeby jego zapytanie czytało jak najmniej bajtów? 🤔

💬 Odpowiedzi na czacie lub na żywo!